13 TEKNIK PENGUJIAN PERANGKAT LUNAK

TEST CASE, WHITE BOX, BLACK BOX

CM 209 Rekayasa Perangkat Lunak

Dr. Eng. Munawir, S.Kom., M.T
Universitas Pendidikan Indonesia
Program Studi Teknik Komputer

Latar Belakang

- Pengujian P/L adalah elemen kritis dari jaminan kualitas P/L dan merupakan review puncak terhadap spesifikasi, desain dan pembuatan program.
- Pengujian P/L menghabiskan upaya 30-40% dari total pekerjaan proyek.
- Untuk proyek yang membahayakan nyawa manusia, biaya pengujian bisa 3-5 X proyek biasa.

Tujuan Pengujian

1. Menjalankan program untuk menemukan error.
2. Test case yang bagus adalah yang memiliki kemungkinan terbesar untuk menemukan error yang tersembunyi.
3. Pengujian yang sukses adalah yang berhasil menemukan error yang tersembunyi.

Prinsip Pengujian

- Harus bisa dilacak hingga sampai ke kebutuhan customer.
- Harus direncanakan sejak model dibuat.
- Prinsip Pareto: 80% error uncovered.
- Dari lingkup kecil menuju yang besar.
- Tidak bisa semua kemungkinan diuji.
- Dilakukan oleh pihak ketiga yang independen.

Testablility

- Kemudahan untuk diuji.
- Karakteristiknya:
  - Operability: mudah digunakan.
  - Observability: mudah diamati.
  - Controlability: mudah dikendalikan.
  - Decomposability: mudah diuraikan.
  - Simplicity: lingkup kecil, semakin mudah diuji.
  - Stability: jarang berubah.
  - Understandability: mudah dipahami.

Desain Kasus Pengujian

- Black box testing
  - Memastikan fungsional P/L berjalan.
  - Kesesuaian input dengan output.
  - Tidak memperhatikan proses logic internal.
- White box testing
  - Pengamatan detail prosedur.
  - Mengamati sampai level percabangan kondisi dan perulangan.

White Box Testing

- Structural Testing or Logic-driven Testing or Glass Box Testing
- Yang dibutuhkan > **Source code**
- Menguji lebih “dekat” tentang detail prosedur perangkat lunak.
- Yang diselidiki: *logical path* (jalur logika) perangkat lunak

Mengapa "Source code"?

- Dengan source code, dapat dilakukan pengujian tentang:
  - Structural Testing process
  - Program Logic-driven Testing
  - Design-based Testing
  - Examines the internal structure of program

Jalur Logika

- Conditions
  - If .. Then ..
  - If .. Then .. Else ..
  - If .. Then .. Else if .. Then ..
  - Case .. Of ..
- Loop
  - While .. Do ..
  - Repeat .. Until ..
  - For .. To .. Do ..

Get a long with them ;)

Contoh Test Case

Generate test cases to cover each statement

```
x: 1..1000;

1   INPUT-FROM-USER(x);
    If (x <= 300) {
2           INPUT-FROM-FILE(BALANCE);
            If (x <= BALANCE)
3                   GiveMoney x;
4           else Print "You don't have $x in your account!!"}
    else
5           Print "You cannot withdraw more than $300";
6   Eject Card;
```

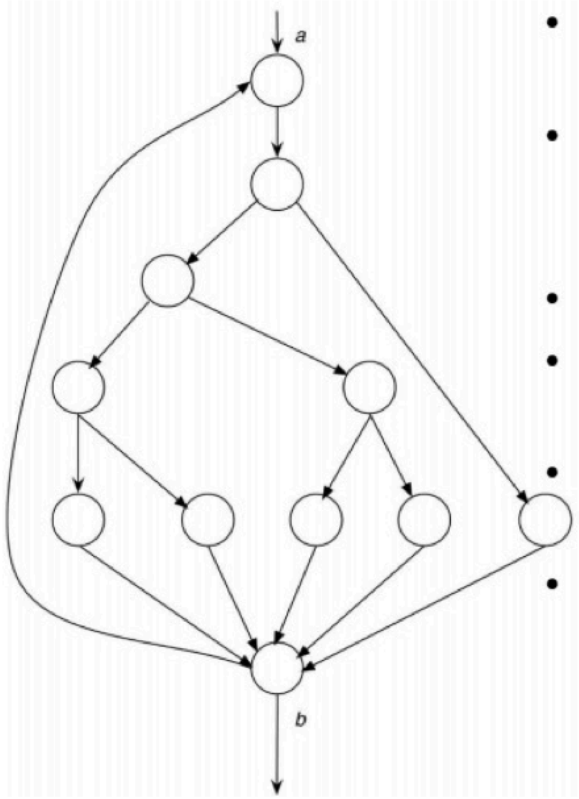
Keuntungan White Box

- Menghasilkan program yang benar dan sempurna 100%, karena:
  - Mengerjakan seluruh keputusan logika
  - Mengerjakan seluruh loop (sesuai batas)
  - Menjamin seluruh jalur independen dalam modul dikerjakan minimal 1x
  - Mengerjakan seluruh data internal yang menjamin validitas

- **Syarat:**
  - Mendefinisikan semua logical path
  - Membangun kasus untuk pengujian
  - Mengevaluasi hasilnya
  - Menguji secara **menyeluruh**

Kekurangan

- Pengujian secara menyeluruh justru menimbulkan masalah sumber daya
- Program yang kecil bisa menghasilkan banyak sekali jalur logika, contoh:



- Terdapat 1 (satu) buah loop yang berulang sebanyak 20 kali
- Di dalamnya terdapat nested if yang terdiri dari 4 (empat) pernyataan if..then..else
- Terdapat 5 jalur
- Asumsi keputusan saling independen
- Jumlah logic path = $5^{20}+5^{19}+5^{18}+...+5^{0}$
- Setara 10^{14} jalur

  10^{14} = 100.000.000.000 (seratus triliun) logic paths

Masih “Kekurangan”

- Setiap jalur harus diuji secara “manual” untuk dibuktikan kebenarannya.
- Manual:
  - Menulis
  - Mengeksekusi
  - Memverifikasi hasil
- 1 jalur logika = 1 kegiatan manual
- Jika 1 manual dilakukan selama 5 menit, Berapa lama waktu yang diperlukan untuk pengujian?

"Bukankah *black box testing* jauh lebih cepat dan lebih mungkin dilakukan?"

- Jawaban terletak pada ketidak-sempurnaan perangkat lunak:
  - Adanya kesalahan logika (logic errors)
    65 < x <=75 > if (x < 65) AND (x <= 75) then…
  - Adanya ketidaksesuasian asumsi (incorrect assumptions)
    1 bulan = 30 hari >bulan Februari? Juli? dst…
  - Adanya kesalahan menulis kode (typographical errors)
    volume:= P* L* T; > polume:=P * L* T;..

Mungkinkah dilakukan pengujian White Box?

- Ya!
- Tidak dilakukan secara menyeluruh.
- Cukup dilakukan pada jalur logika yang penting.
- Kombinasikan dengan *black box testing*.

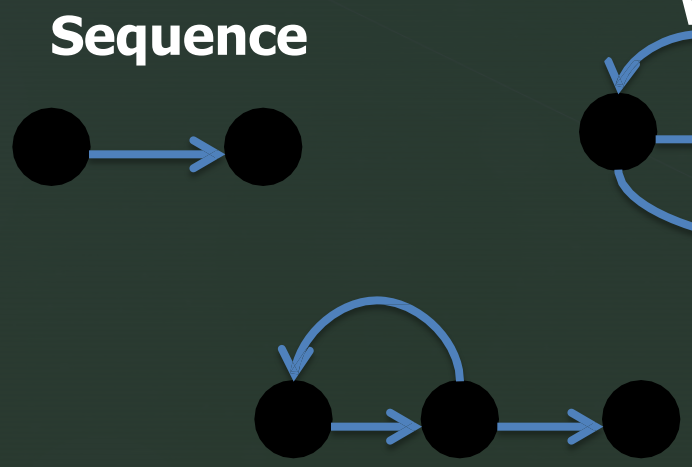
Metode White Box | Basis path testing

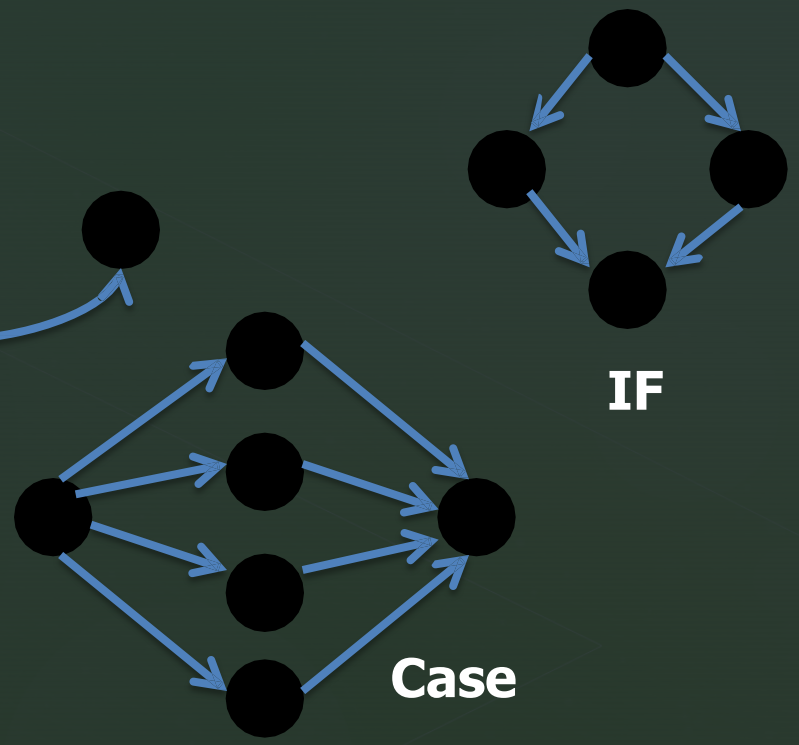
- Diusulkan oleh Tom McCabe pada tahun 1976
- Digunakan untuk mendapatkan ukuran kompleksitas logika
- Ukuran ini dijadikan sebagai panduan untuk menentukan jalur-jalur utama untuk dieksekusi.

Notasi

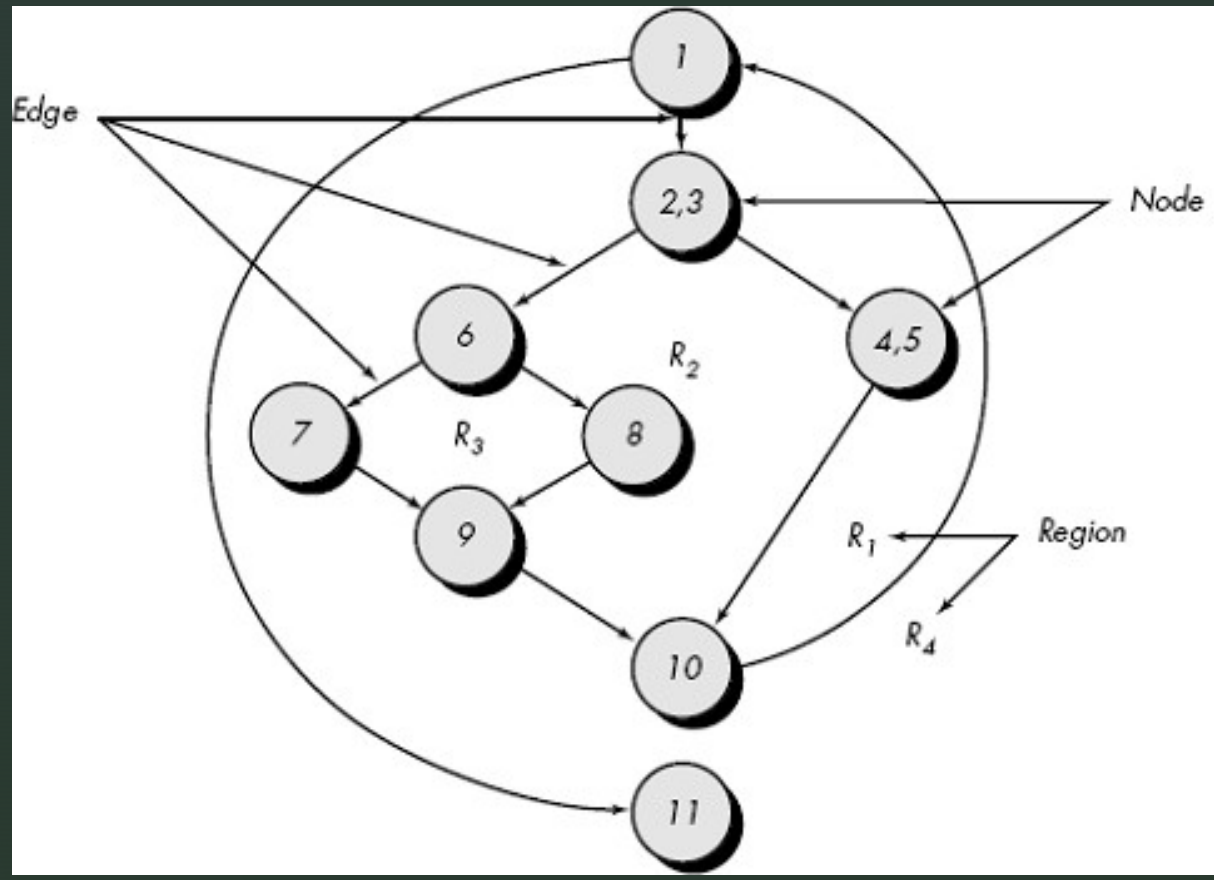
Lingkaran (flow graph node)
Menyatakan satu atau beberapa statement prosedural

Panah (edges / links)
Menyatakan aliran kendali (sama dengan panah flowchart)





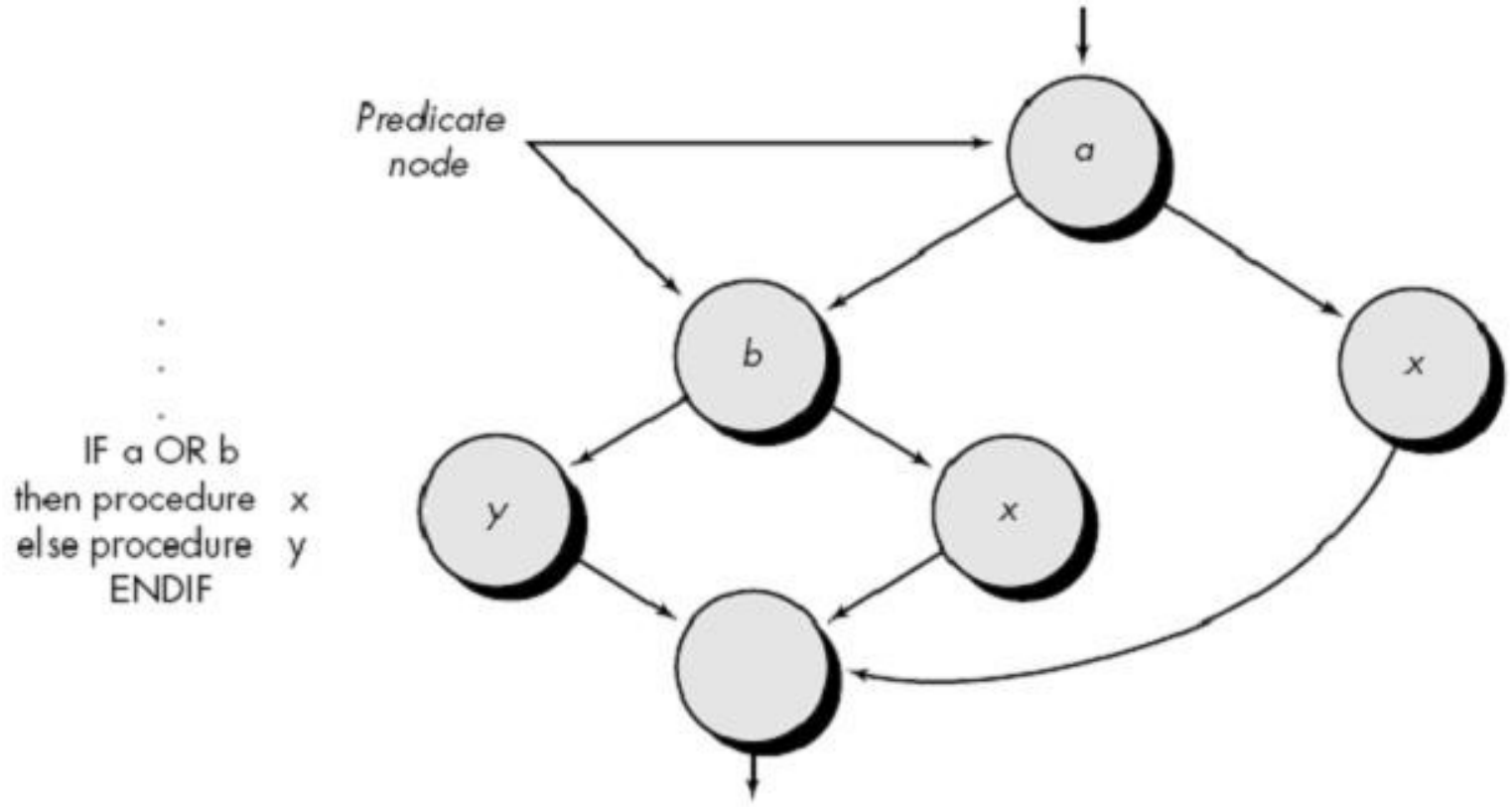
Regions



- Adalah suatu area yang dibatasi oleh edges dan nodes
- Pada saat menghitung regions, area di luar graph ikut ditambahkan

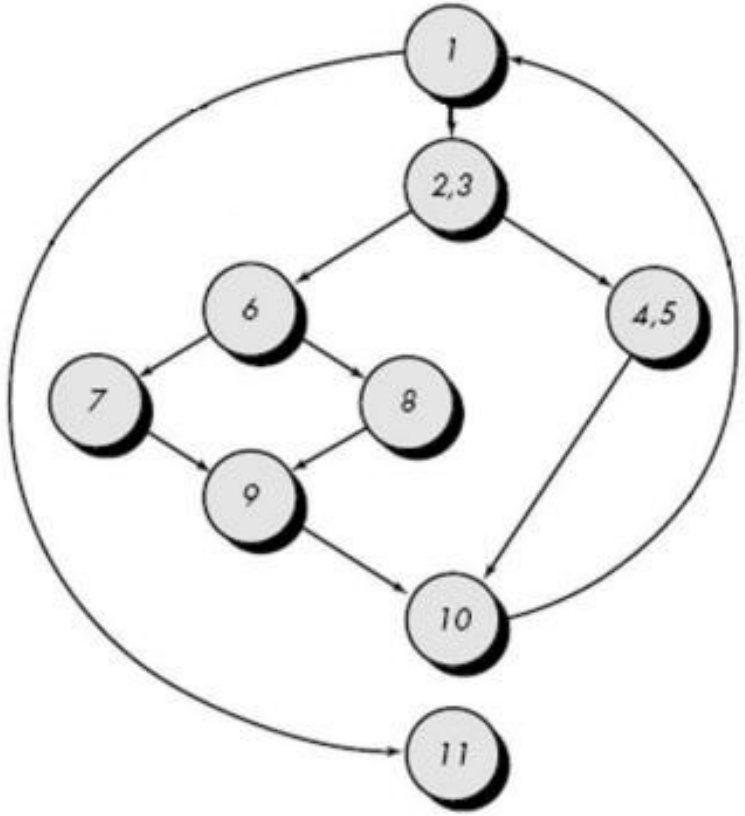
Compound Conditions

- Kondisi majemuk terjadi apabila satu atau lebih dari operator Boolean (AND, OR, NOR, NAND)muncul pada pernyataan kondisi.
- Setiap *node* yang terdapat kondisi disebut ***predicate node***



Independent Path

- Adalah jalur yang melintasi minimal satu kumpulan pernyataan baru atau sebuah kondisi baru pada program.



Path 1 : 1 – 11
Path 2 : 1 – 2 – 3 – 4 – 5 – 10 – 1 – 11
Path 3 : 1 – 2 – 3 – 6 – 8 – 9 – 10 – 1 – 11
Path 4 : 1 – 2 – 3 – 6 – 7 – 9 – 10 – 1 – 11

1-2-3-4-5-10-1-2-3-6-8-9-10-1-11 **bukan** Independent path, karena merupakan kombinasi dari *paths* yang sudah didefinisikan.

Cyclomatic Complexity

- Untuk menentukan banyaknya *independent path* yang merupakan *basis path*
- Rumus:

$$V(G) = E - N + 2$$

$$V(G) = P + 1$$

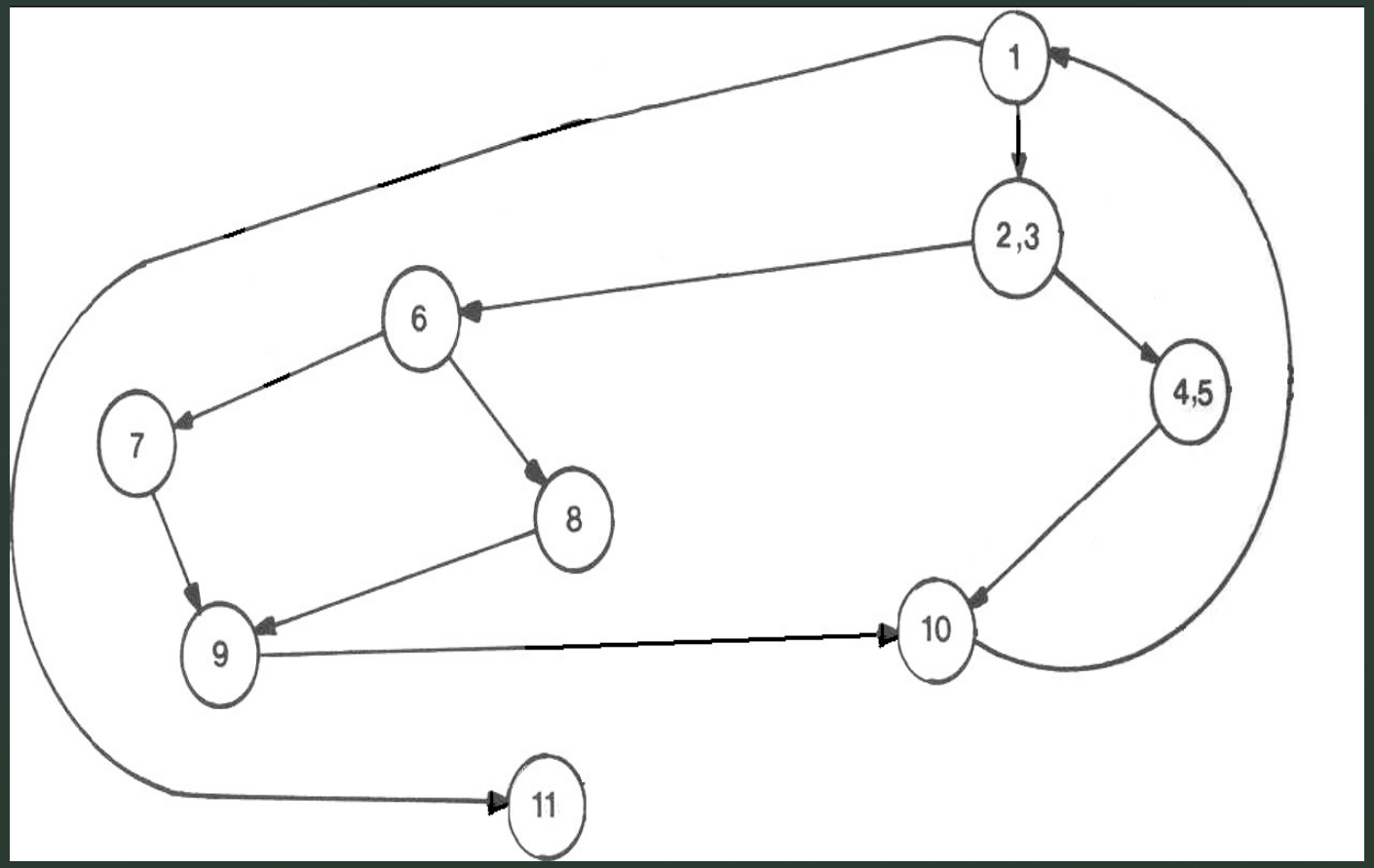
dimana:

E: Edge N :

Node

P: Predicate node

Contoh:



Tentukan independen path?

Jawab :

Dari gambar:

- Path 1 = 1 - 11
- Path 2 = 1 - 2 - 3 - 4 - 5 - 10 - 1 - 11
- Path 3 = 1 - 2 - 3 - 6 - 8 – 9 - 10 - 1 - 11
- Path 4 = 1 - 2 - 3 - 6 - 7 - 9 - 10 - 1 - 11
- Path 1,2,3,4 yang telah didefinisikan diatas merupakan basis set untuk diagram alir.

cyclomatic complexity

- Flowgraph mempunyai 4 region
- V(G) = 11 edge - 9 node + 2 = 4
- V(G) = 3 predicate node + 1 = 4
- Jadi cyclomatic complexity untuk flowgraph adalah 4

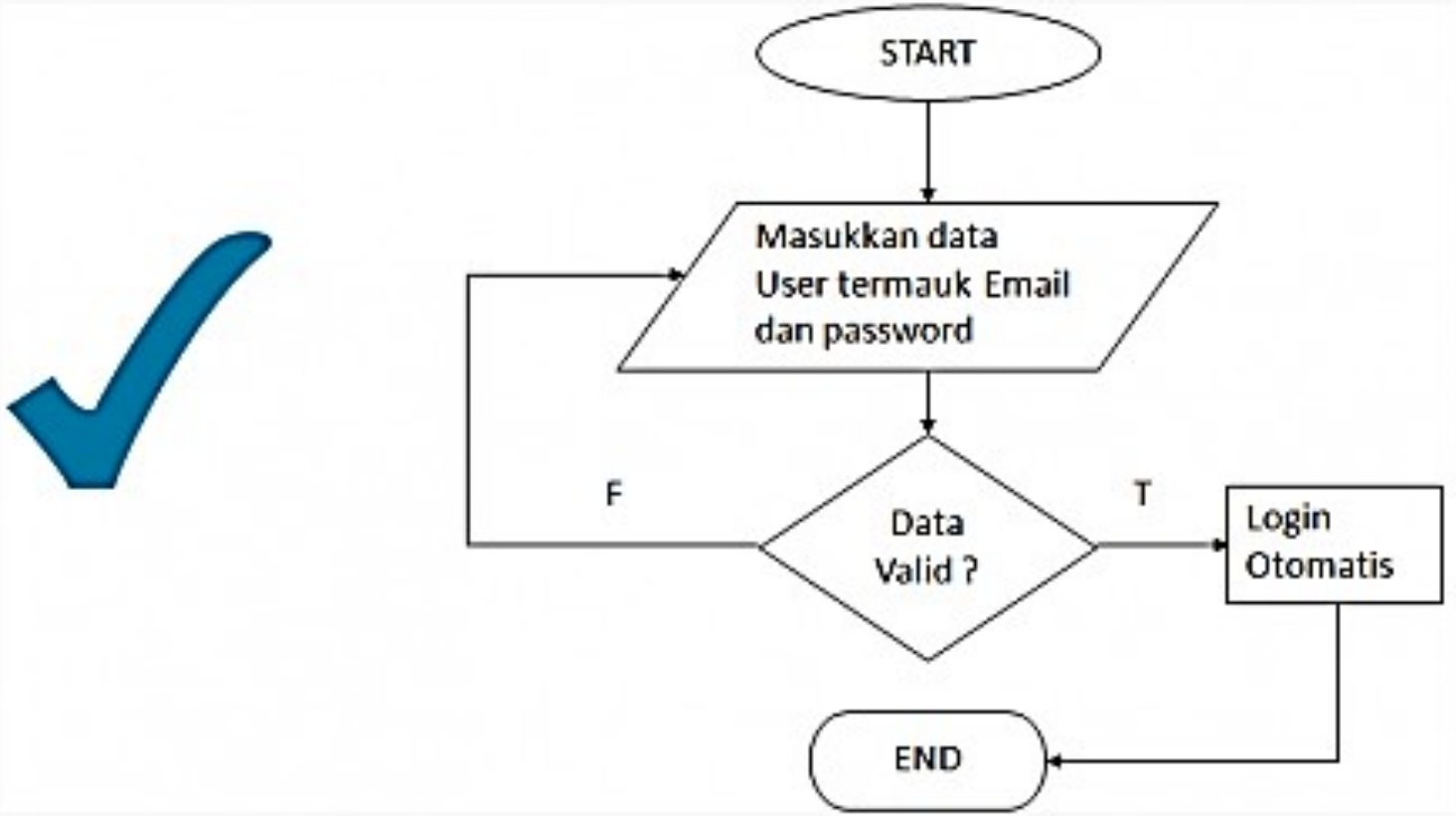
Contoh pengujian WhiteBoX

Fastncheap merupakan sebuah toko yang khusus menjual Khusus barang elektronik, mulai komputer, LCD, GPS, Laptop, dan asesoris lainnya. Fastncheap juga menyediakan toko atau website secara online. Di dalam website ini terdapat beberapa kategori dimana di setiap topik diberikan penjelasan secara rinci baik spesifikasi, kelebihan dan kelemahan terhadap produk yang dikehendaki sehingga menjadi bahan pertimbangan sebelum membeli produk tersebut.

Toko Fast & Cheap telah menggunakan Sistem Web Aplikasi Penjualan Online dimana produk disana sudah bisa dibeli secara online. Sistem disini harus bisa menampilkan Informasi Penjualan Komputer pada Toko Toko Fast & Cheap (FNC) secara online yang mempunyai beberapa cabang di Indonesia

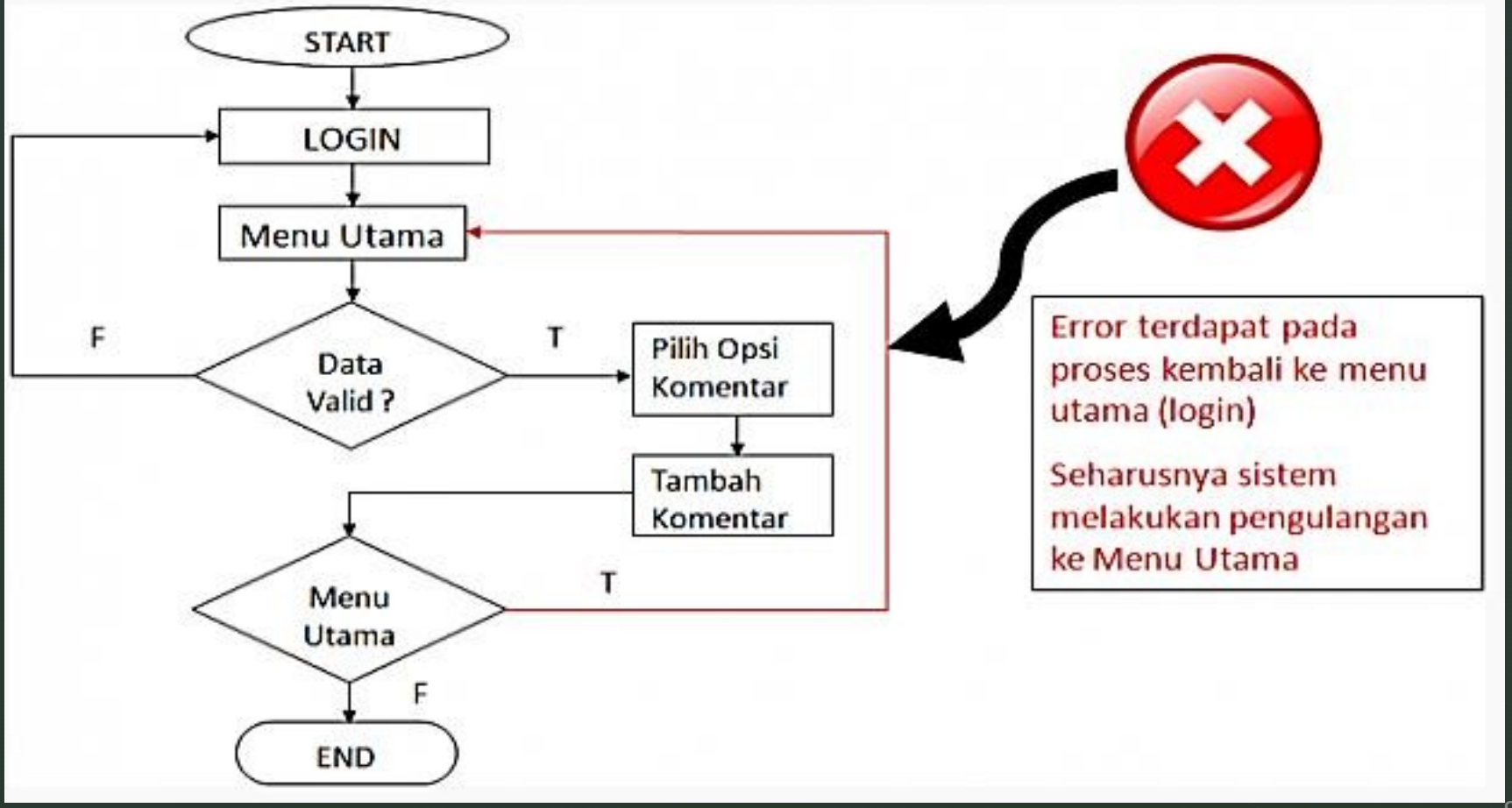
Contoh pengujian White Box

Seperti yang diatas, pertama-tama kami menguji form registrasi anggota



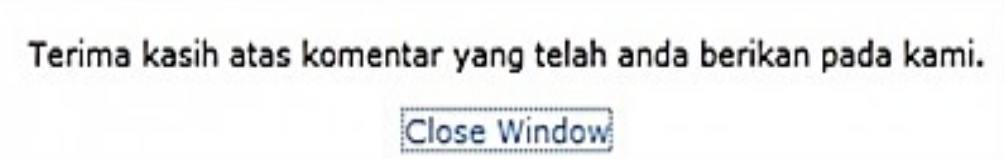
Contoh pengujian White Box

Yang kedua kami menguji form tambah komentar



Contoh pengujian White Box

screenshot:

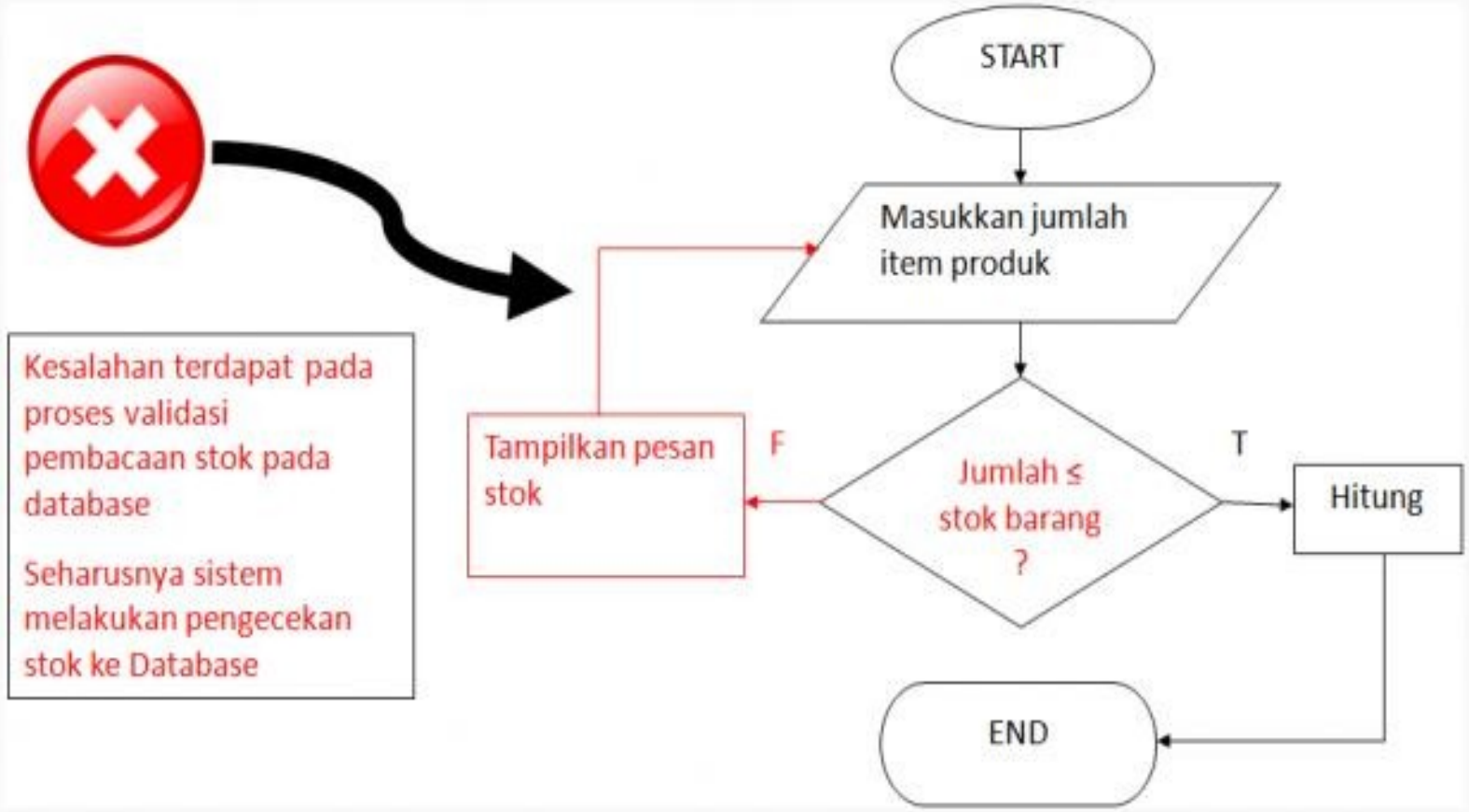


Fungsi tombol "Close Window" setelah memberikan komentar tidak berjalan sesuai fungsinya.

Diperkirakan link yang terdapat pada Tulisan "Close Window" sama dengan link pada posting komentar sehingga halaman tidak berpindah

Contoh pengujian White Box

Yang ketiga kami menguji form shoping cart



Contoh pengujian White Box

screenshot :

Tidak terdapat validasi jumlah stok produk terhadap database sehingga bisa memasukkan angka jumlah produk yang sangat besar

Contoh pengujian White Box

Maintenance yang bisa dilakukan

- Pembersihan, pengawasan, dan pengecekan komentar yang masuk selama proses update produk
- Pengecekan kebenaran dan kecocokan database sistem terhadap produk yang ditampilakan
- Perbaikan tampilan sistem web agar lebih mudah diakses
- Pengecekan terhadap domain yang digunakan
- Pengecekan keamanan program (pembelian secara curang atau order dari Negara tertentu yang ilegal) yang digunakan sistem seiring perkembangan jaman
- Pengecekan terhadap pihak penerima pembayaran online
- Peningkatan kualitas layanan searching (pencarian)

Terlalu sering maintenance juga tidak baik karena berdampak buruk terhadap kenyamanan pelanggan

Contoh pengujian White Box

Fitur-fitur yang bisa ditambahkan

- Forum per topik, sehingga terdapat pertimbangan-pertimbangan dalam hal menentukan pilihan dalam membeli produk yangs sedang di bahas dalam forum tersebut
- Terdapat fasilitas diskon untuk member dengan ketentuan sedemikian rupa sehingga bisa memperbanyak pelanggan.
- Fitur pengganti bahasa. Sehingga pelanggan dari dalam negeri dan luar negeri bisa juga menggunakan web tersebut.
- Fitur Cetak dokumen bagi pelanggan yang ingin mencetak informasi tentang produk ataupun di forum (jika sudah ada)
- Sistem agar bisa diakses lewat mobile
- Penambahan fitur facebook, twitter, blog dsb. Untuk dapat lebih memudahkan dalam hal promosi.

Teknik-teknik *Black Box Testing*

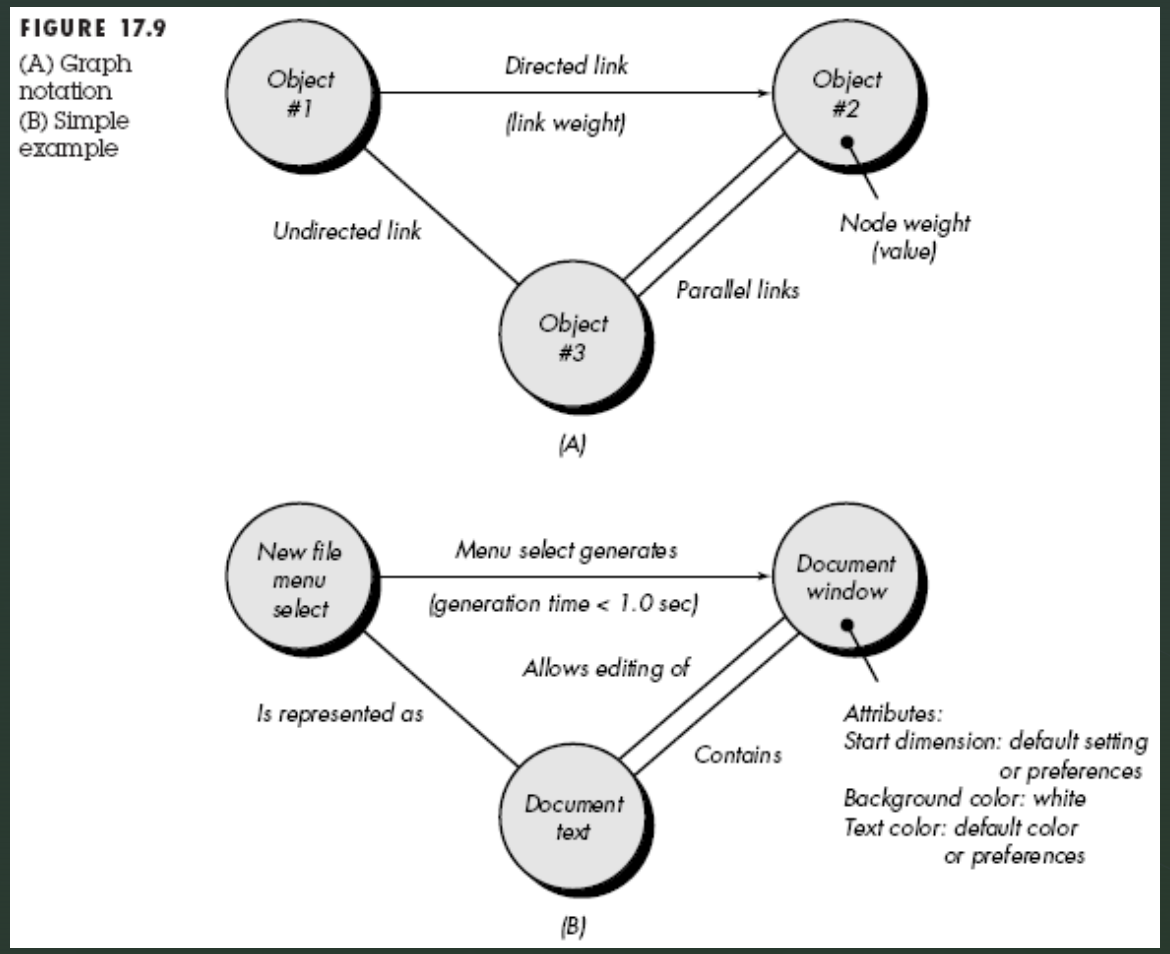
Ada beberapa teknik yang biasanya digunakan untuk menguji perangkat lunak. Berikut ini adalah teknik-tekniknya:

- ***All pair testing***
  - Teknik *all pair testing* ini dikenal juga dengan *pairwise testing*. Pengujian ini digunakan untuk menguji semua kemungkinan kombinasi dari seluruh pasangan berdasarkan input parameternya.
- *Boundary value analysis*
  - Teknik ini berfokus pada pencarian error dari luar atau sisi dalam perangkat lunak
- *Cause-effect graph*
  - Teknik *cause-effect graph*. Teknik pengujian ini menggunakan grafik sebagai patokannya. Grafik ini menggambarkan relasi antara efek dan penyebab dari error.

Teknik-teknik *Black Box Testing*

- *Equivalence partitioning*
  - Teknik ini bekerja dengan cara membagi data *input* dari beberapa perangkat lunak menjadi beberapa partisi data.
- *Fuzzing*
  - Fuzzing merupakan teknik pencarian bug dalam perangkat lunak dengan memasukan data yang tidak sempurna.
- *Orthogonal array testing*
  - *orthogonal array testing*. Teknik ini digunakan jika input berukuran kecil, akan tetapi cukup berat jika digunakan dalam skala yang besar.
- *State transition*
  - Terakhir adalah *state transition*. Teknik ini berguna untuk melakukan pengujian terhadap mesin dan navigasi dari UI dalam bentuk grafik

Black Box Testing – Graph Based



Black Box Testing – Equivalence Partitioning

1. If an input condition specifies a *range*, one valid and two invalid equivalence classes are defined.
2. If an input condition requires a specific *value*, one valid and two invalid equivalence classes are defined.
3. If an input condition specifies a member of a *set*, one valid and one invalid equivalence class are defined.
4. If an input condition is *Boolean*, one valid and one invalid class are defined.

- Contoh: Input NPM dalam SIAMIK
  - Jika dikosongi?
  - Jika diisi dengan format yang salah?
  - Jika diisi dengan NPM yang benar?

Black Box Testing – Analisa Nilai Batas

1. Menguji untuk input di sekitar batas atas maupun bawah sebuah range nilai yang valid.
2. Menguji nilai maksimal dan minimal.
3. Menerapkan (1 & 2) untuk output.
4. Menguji batas struktur data yang dipakai. Misal ukuran array.

Black Box Testing – Perbandingan

- Spesifikasi kebutuhan yang sama dimungkinkan menghasilkan aplikasi/ perangkat lunak yang berbeda.
- Skenario pengujian pada aplikasi yang demikian bisa digunakan untuk skenario pengujian aplikasi serupa yang lain.

Skenario Pengujian Khusus

- Pengujian GUI.
- Pengujian arsitektur client/ server.
- Pengujian dokumentasi dan fasilitas bantuan.
- Pengujian sistem waktu nyata.

- **Keuntungan**
  - Ketika *behavioral testing* digunakan dalam pengujian perangkat lunak, ada beberapa keuntungan yang dapat diperoleh dari pengujian tersebut. Berikut ini adalah keuntungannya:
  - Penguji tidak harus memiliki pengetahuan tentang suatu bahasa pemrograman.
  - Pengujian dilakukan berdasarkan sudut pandang pengguna. Hal tersebut dilakukan agar dapat menemukan inkonsistensi dalam perangkat lunak.
  - Pengembang dan penguji memiliki ketergantungan satu dengan yang lainnya.
  - Penguji tidak perlu memeriksa kode.
  - Memungkinkan penguji dan pengembang bekerja secara independen tanpa mengganggu proses kerja satu sama lain.

- **Kekurangan**

  Berikut ini adalah beberapa kekurangannya:

  - Memiliki kemungkinan kesalahan tidak terdeteksi karena kurang teliti dan tidak adanya pengetahuan teknis.
  - Ada bagian back-end yang tidak diuji sama sekali.
  - Kemungkinan pengujian dilakukan kembali oleh programmer

Contoh Pengujian Black Box

Pengujian Black Box Testing – Black box testing adalah pengujian dilakukan dengan cara memberikan sejumlah input pada program. Input tersebut kemudian diproses sesuai dengan kebutuhan fungsionalnya untuk melihat apakah program aplikasi dapat menghasilkan output yang sesuai dengan yang diinginkan dan sesuai pula dengan fungsi dasar dari program tersebut.

Contoh Pengujian Black Box Testing

Berikut ini contoh hasil pengujian black Box Testing pada aplikasi website TV local .

Nama Aplikasi:			**Tanggal Pengujian:**		
			Tester :		
No.	**Halaman Yang Diuji**	**Aksi Aktor**	**Reaksi Sistem**		**Hasil**
			Benar	**Salah**	
A.	**Halaman Sistem Admin**				
1.	Halaman Awal Admin	Pilih menu *login*.	Masuk ke halaman *login* admin	Tidak masuk ke halaman *login* admin	Sesuai ( ) Tidak Sesuai ( )
2.	*Login* Admin	Memasukkan *username* dan *password*	Masuk ke halaman utama *website* admin	Akan kembali ke halaman *login* dan keterangan (*username* atau *password* salah)	Sesuai ( ) Tidak Sesuai ( )
3.	*Logout*	Plih menu *logout*	Akan melakukan *logout* atau keluar dari sistem dan tampil ke halaman awal admin	Tidak melakukan *logout*	Sesuai ( ) Tidak Sesuai ( )
4.	Data pelanggan	Pencarian data pelanggan	Akan menampilkan data pelanggan yang dicari	Data pelangga yang dicari tidak ditemukan	Sesuai ( ) Tidak Sesuai ( )
5.	Data Iklan	Menambahkan data iklan dengan memasukkan semua data iklan	Data iklan akan bertambah	Menampilkan pesan kesalahan jika data ada yang tidak diisi	Sesuai ( ) Tidak Sesuai ( )
		Mengubah data iklan	Data iklan terbaru akan berhasil	Menampilkan pesan	Sesuai ( )

Contoh Pengujian Black Box

B.	**Halaman Sistem Pelanggan**				
1.	*Register*	Mengisi data dengan lengkap	Berhasil membuat akun	Tidak berhasil membuat akun atau menampilkan pesan kesalahan jika data ada yang tidak diisi	Sesuai ( ) Tidak Sesuai ( )
2.	Pemasangan Iklan	Memasang Iklan Video dengan mengisi seluruh data dan juga link video	Pemasangan iklan video berhasil	Tidak berhasil memasang iklan atau menampilkan pesan kesalahan jika data ada yang tidak diisi	Sesuai ( ) Tidak Sesuai ( )
		Memasang Iklan Poster dengan mengisi seluruh data dan mengunggah poster	Pemasangan iklan poster berhasil	Tidak berhasil memasang iklan atau menampilkan pesan kesalahan jika data ada yang tidak diisi	Sesuai ( ) Tidak Sesuai ( )
3.	Pembayaran	Mengunggah bukti	Bukti pembayaran berhasil	Bukti pembayaran tidak	Sesuai ( )

STRATEGI PENGUJIAN P/L

- Membahas langkah-langkah yang harus dikerjakan sebagai bagian dari pengujian.
- Kapan dilaksanakan? Berapa usaha, waktu dan sumber daya yang digunakan?
- Meliputi: perencanaan, desain test case, pelaksanaan, koleksi data dan evaluasi.

Kaidah Umum Pengujian

- Dimulai dari pengujian tingkat komponen menuju integrasi.
- Titik yang berbeda dimungkinkan memakai teknik pengujian yang berbeda.
- Pengujian dilakukan oleh developer dan (untuk proyek yang besar) tim independen.
- Testing dan debugging adalah berbeda. Namun debugging pasti berkaitan dengan strategi testing apapun.

Strategi Pengujian

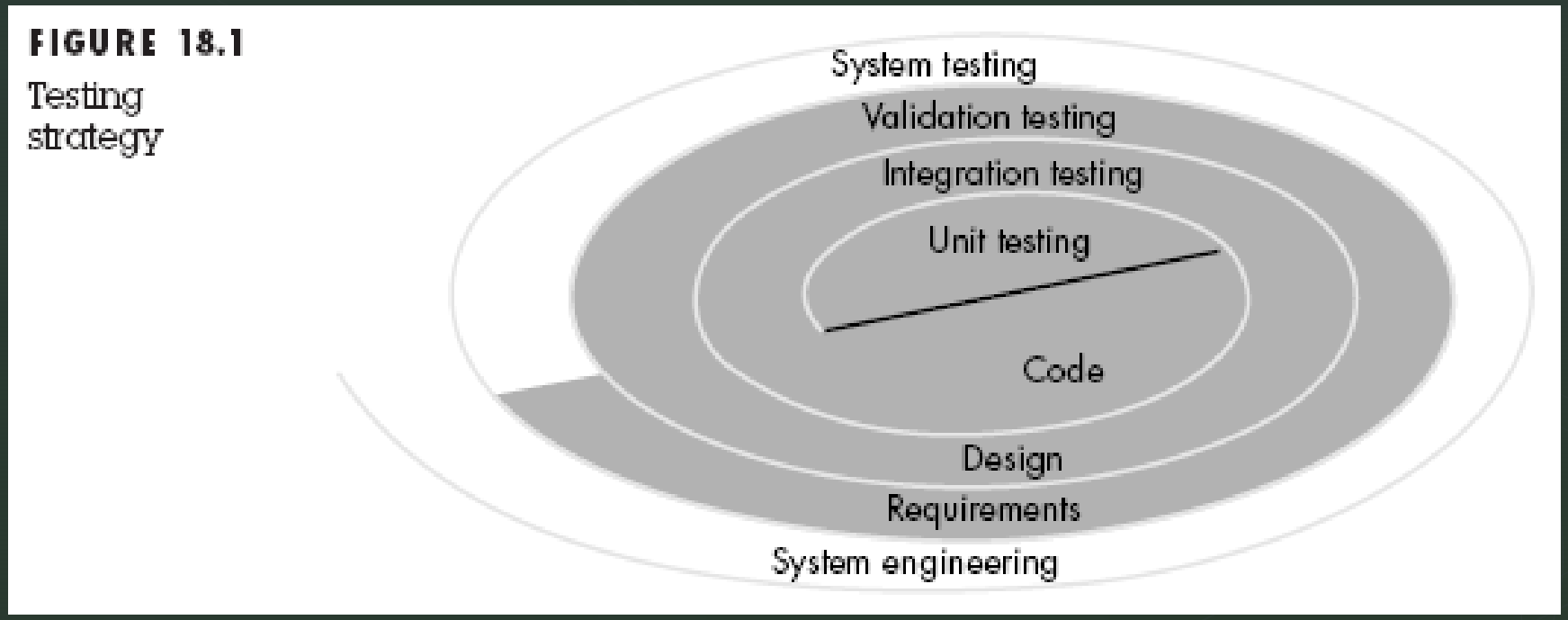
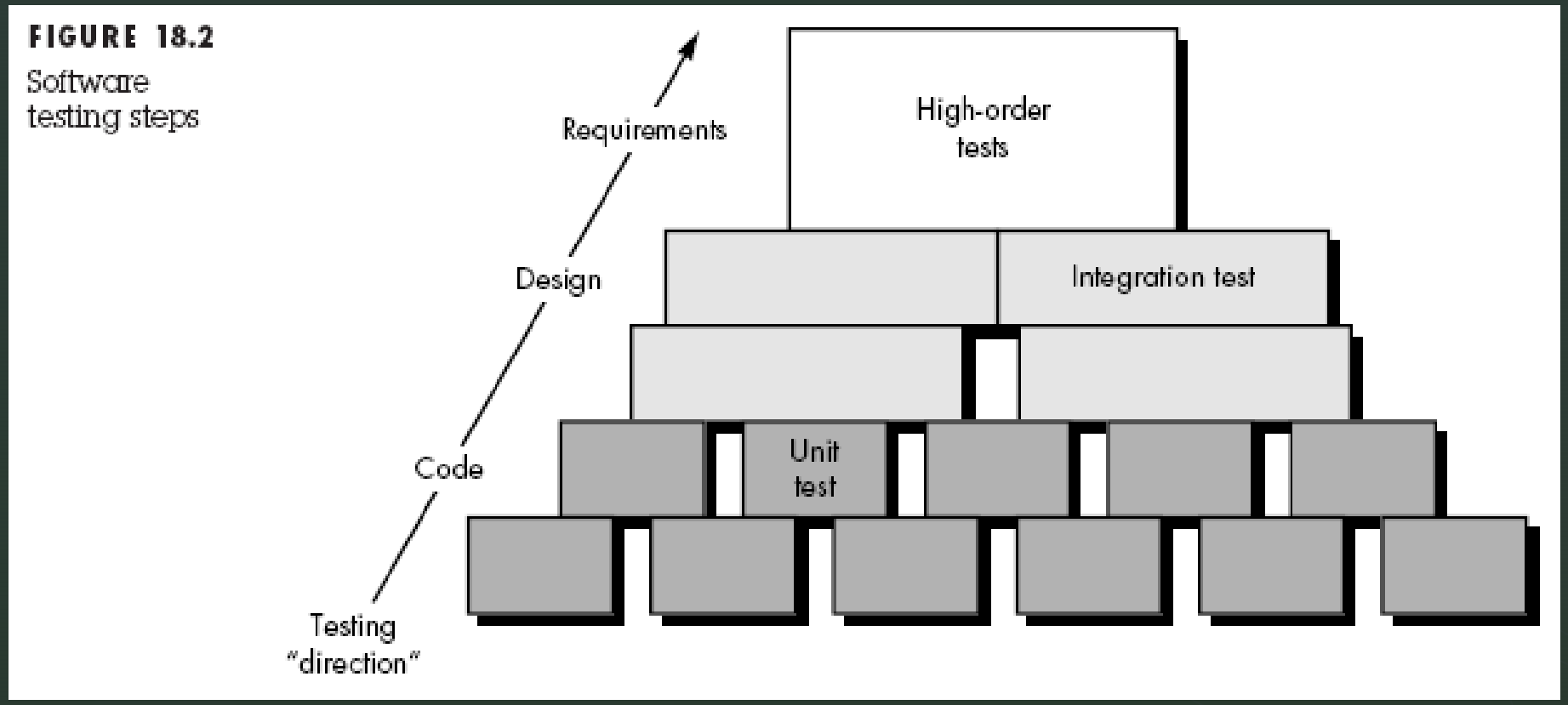


FIGURE 18.1
Testing strategy

Langkah Pengujian



Unit Testing

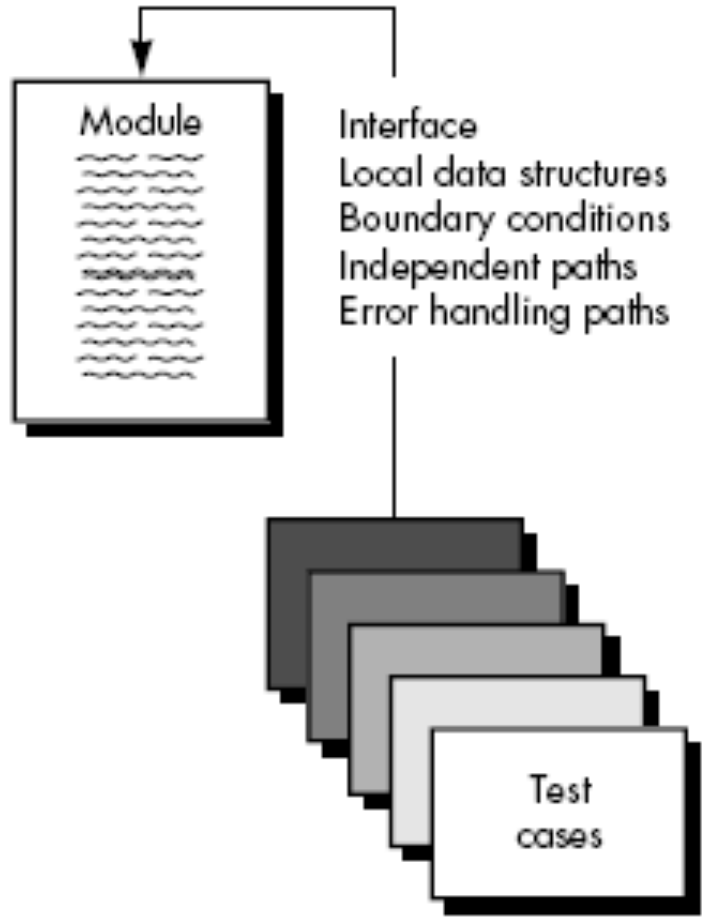


FIGURE 18.4
Unit test

Integration Testing

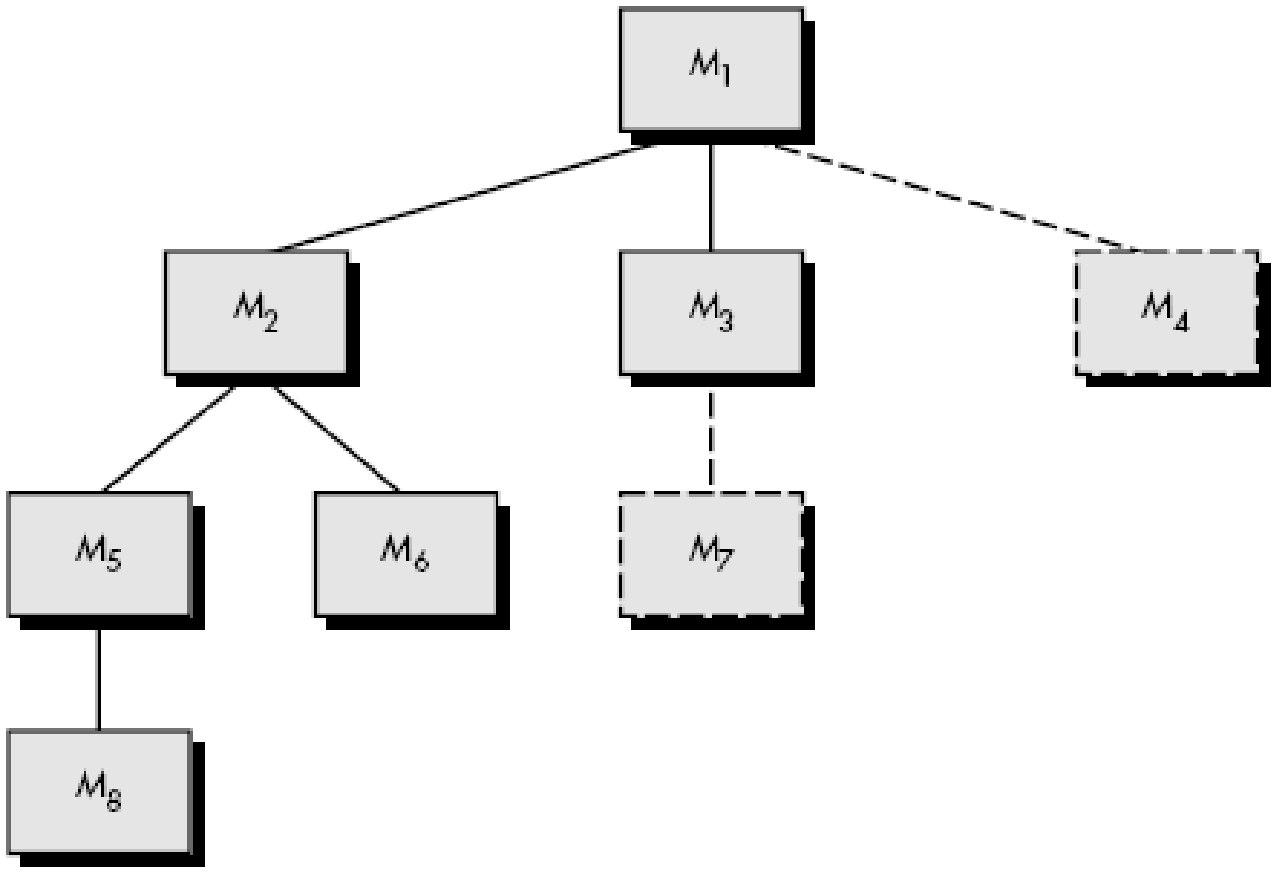


FIGURE 18.6
Top-down integration

Integration Testing

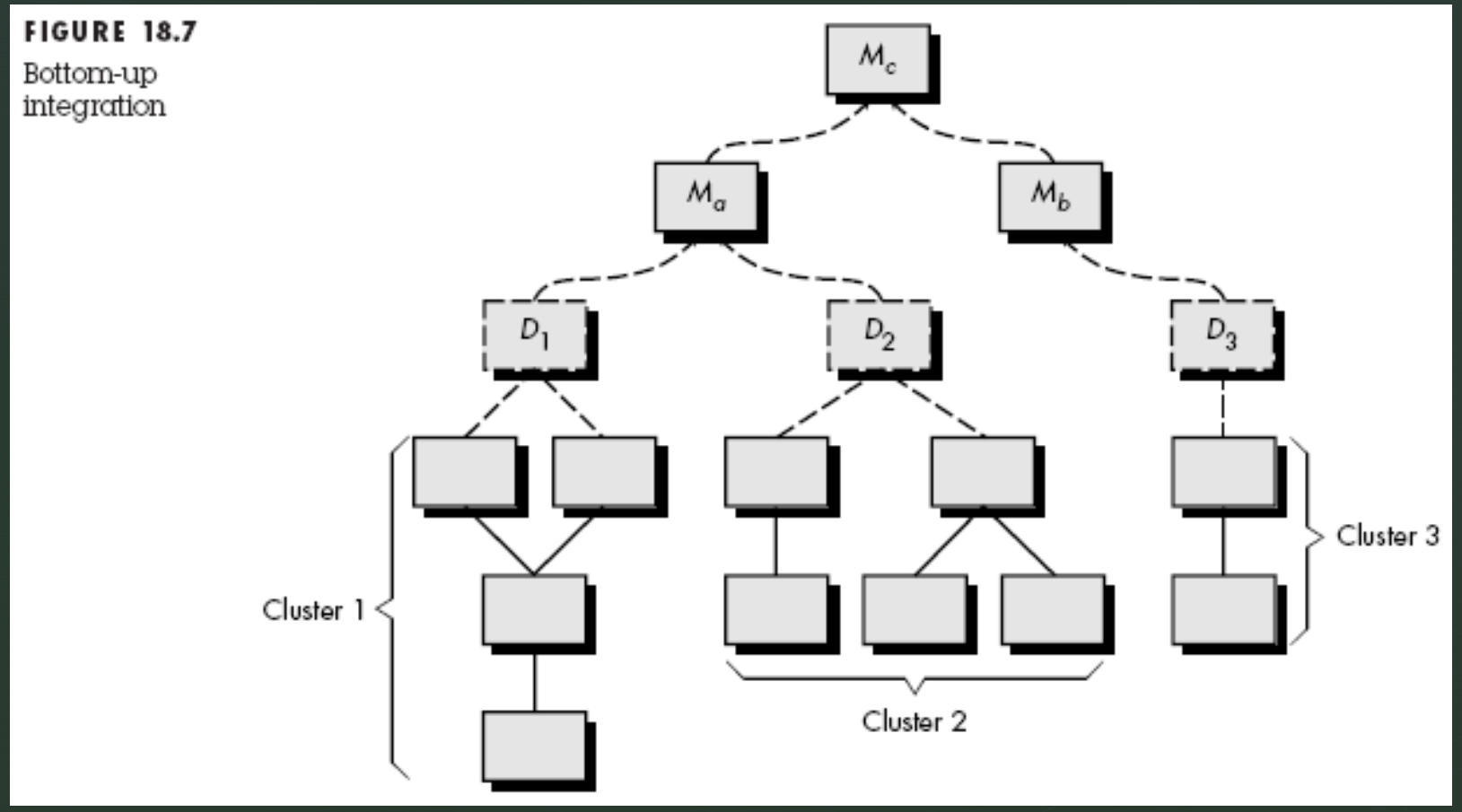


FIGURE 18.7
Bottom-up integration

Integration Testing

- Regression testing: dilakukan pengujian setiap kali ada modul baru yang diintegrasikan atau ada modul yang berubah.
- Smoke testing: test daily, untuk proyek jenis kritis-waktu.

Validation Testing

- Disebut sukses jika fungsi P/L dapat diterima oleh customer (berdasarkan dokumen SKPL).
- Alpha test: dilakukan di tempat developer oleh customer pada lingkungan yang terkendali.
- Beta test: dilakukan di tempat customer tanpa melibatkan developer pada lingkungan yang tak terkendali.

System Testing

- Meguji sistem berbasis komputer secara menyeluruh, termasuk juga hubungannya dengan sistem yang lain.
- Diantaranya:
  - Recovery testing, jika system failure.
  - Security testing, jika terjadi serangan.
  - Stress testing, terhadap jumlah, frekuensi dan volume pekerjaan.
  - Performance testing, untuk mengukur pemakaian sumber daya.

Debugging

- Memperbaiki error yang ditemukan pada saat testing (yang sukses).
- Kaidah dasar sebelum debug:
  - Apakah penyebab bug dihasilkan kembali oleh bagian program yang lain?
  - Apakah bug selanjutnya yang mungkin muncul jika bug diperbaiki?
  - Apa yang bisa dilakukan untuk mencegah bug terjadi untuk pertama kalinya?